Return-to-libc Attacks

Dogăreci Bianca-Alexandra, Iscru Bianca, Roman Mateea
Computer Science Department, University of Bucharest, Romania

Abstract

Return-to-libc attacks extend classical buffer overflow techniques by bypassing non-executable memory protections. This paper examines the challenges of exploiting buffer overflows under DEP/NX and shows how attackers adapt to architectural constraints. We first apply Krahmer's borrowed code chunks technique on a modern x86-64 system to demonstrate how return-to-libc attacks can overcome NX protection. We then use Wojtczuk's return-into-lib(c) work to explain how the additional hardening introduced by ASLR can be bypassed through techniques such as `dl-resolve`. Together, these approaches show that code reuse attacks remain viable despite modern defenses.

1 Introduction

Buffer overflow vulnerabilities remain a persistent security threat despite decades of defensive innovations. They exploit the lack of bounds checking in memory-unsafe languages such as C and C++, allowing attackers to overwrite critical data and redirect program execution.

Early exploits injected shellcode onto the stack and redirected execution to it. This approach was largely stopped by Data Execution Prevention (DEP) and the NX bit, which enforce the W⊕X policy by separating writable and executable memory. In 1997, Solar Designer showed that attackers could adapt by reusing existing executable code instead of injecting new instructions [1]. This led to return-to-libc attacks, where execution is redirected to existing library functions, particularly from the standard C library (libc), such as `system()` or `execve()`.

The transition from 32-bit x86 to x86-64 introduced a further challenge: while 32-bit systems pass function arguments on the stack, the x86-64 System V ABI passes arguments in registers. This change makes classical return-to-libc techniques ineffective. Krahmer [2] addresses this problem by showing how short instruction sequences ending in `ret` can be reused to control register values, restoring the ability to perform code reuse attacks.

However, this technique assumes that code locations are predictable. Modern operating systems weaken this assumption through Address Space Layout Randomization (ASLR). Wojtczuk's work on advanced return-into-lib(c) exploits [3] studies how such defenses affect return-to-libc attacks and how attackers can recover reliability. Together, these works show how attackers both construct usable code reuse attacks and keep them reliable despite address randomization.

2 Technical description of the problem

2.1 Control-flow hijacking

A stack-based buffer overflow allows an attacker to overwrite the saved return address of a function. This ability to redirect control flow is the fundamental primitive used by return-to-libc and other code reuse attacks.

However, control-flow hijacking alone is not sufficient for successful exploitation. The attacker must also ensure that execution reaches useful code with a valid program state, including correctly initialized function arguments and a consistent stack layout.

2.2 Non-executable memory

The introduction of NX protection prevents the execution of injected code from writable memory regions, such as the stack. As a result, attackers can no longer rely on executing shellcode placed

directly in overflowed buffers.

This restriction forces attackers to reuse existing executable code is already present in the program or linked libraries. While this bypasses NX, it significantly constrains what an attacker can execute and places stricter requirements on how control flow must be redirected.

2.3 Impact of x86-64 Calling Conventions on Exploitation

The transition from 32-bit x86 to x86-64 introduces a fundamental architectural change: the first function arguments are passed in registers rather than on the stack. Specifically, the first six integer or pointer arguments are passed using the `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, and `%r9` registers. Consequently, overwriting the return address alone is no longer sufficient to perform a function call with attacker-controlled parameters.

This change means that even if the control flow is successfully redirected to a library function, the function may receive incorrect or uninitialized arguments. Any exploitation technique on x86-64 must therefore address not only control-flow redirection, but also the problem of preparing the required register state.

2.4 Reliability under ASLR

Early defenses against control-flow hijacking have already recognized the importance of address unpredictability. In the PaX Linux kernel patch as analyzed by Nergal in his paper [3], the RANDMMAP feature randomizes the base addresses of the stack and of memory-mapped shared objects, such as the dynamic linker and libc, on each program execution. As a result, exploits that assume fixed locations for library code or stack frames can no longer be reused reliably across runs.

Modern operating systems generalize this idea through Address Space Layout Randomization (ASLR), which extends randomization to additional memory regions. Under ASLR, exploits that rely on hard-coded addresses may fail unpredictably, even when control flow redirection and argument setup are otherwise correct. Ensuring reliable exploitation therefore requires techniques to discover or recover code locations at runtime, making reliability under address randomization a central technical challenge.

3 Attacker Adaptations

3.1 Krahmer: borrowed code chunks on x86-64

On x86-64, a return-to-libc attack must both redirect control flow to existing executable code, since injected code is blocked by NX, and ensure that the target function receives correct arguments through the appropriate registers.

Krahmer observes that most binaries and libraries already contain many short instruction sequences that end in `ret`. A `ret` instruction pops an address from the stack and jumps to it, so that a chain of `ret`-terminated sequences can be executed one after another by preparing a sequence of addresses on the stack.

The key idea is to use "borrowed code chunks" (now commonly called gadgets) that copy attacker-controlled values from the stack into registers. The simplest examples are sequences like "pop register; ret". Each time such a gadget runs, it consumes one 8-byte value from the stack and places it into a chosen register, then continues to the next address on the stack.

This solves the main x86-64 problem: even though function arguments are not passed on the stack, the attacker can still set argument registers by executing gadgets first. After loading the needed registers, the chain jumps to a target function in libc or in the main program.

An important observation in Krahmer's work is that gadgets do not need to start at locations intended by the compiler. On x86, instructions have variable length and the processor does not enforce alignment. As a result, execution can begin at any byte address, including the middle of an existing instruction.

This means that useful instruction sequences may appear unintentionally inside other instructions. By redirecting execution to such byte offsets, an attacker can obtain valid instruction sequences ending in `ret`. This greatly increases the number of available gadgets, especially in large libraries such as libc. Consequently, even programs with small code sections may remain exploitable because they

link against large shared libraries.

3.2 Wojtczuk: Reliability under Address Randomization

Krahmer-style gadget chains assume that the attacker can jump to known code addresses (gadgets in the main binary or in libc). Address randomization breaks this assumption by shifting important mappings between executions. Wojtczuk studies this problem in the PaX kernel patch, where the `RANDMMAP` feature randomizes the base address of the first memory-mapped shared object and places subsequent libraries after it; the stack is randomized as well. This prevents a simple "return-to-libc" payload from being reused across runs, because the absolute address of a target function changes each time.

Wojtczuk also explains why randomization does not automatically imply security. In local attacks, the attacker may be able to view the memory layout of a process through operating system information, such as exposed memory mappings. In some settings, the randomization space can be small enough that repeated trials eventually hit the correct base address. Finally, even under library randomization, the main executable may remain at a fixed location (for non-PIE binaries), which means that attacker-controlled control flow can still reliably reach code in the program itself.

The key contribution in Wojtczuk's paper is a more general method that avoids hard-coding the address of a library function. Instead of jumping directly to a libc function, whose address is unknown under address randomization, the attacker forces execution through the dynamic linker's symbol resolution path (`dl-resolve`).

On the first call to an external function, the execution enters the corresponding PLT stub. The stub performs an indirect jump through the GOT entry. If the symbol has not yet been resolved, the GOT entry points back to the next instruction inside the same PLT stub. This instruction pushes a relocation index onto the stack and then transfers control to a special PLT entry (PLT0), which pushes the `link_map` pointer and invokes the dynamic linker's resolver function.

The resolver then consults ELF metadata structures, including relocation entries in `.rela.plt` (referenced by `JMPREL`), the dynamic symbol table `.dynsym` (referenced by `SYMTAB`), and the dynamic string table `.dynstr` (referenced by `STRTAB`), to resolve the function name to its actual runtime address in a loaded shared library. The resolved address is stored in the Global Offset Table (GOT), so that subsequent calls jump directly to the function without invoking the resolver again.

Wojtczuk's key insight is that the resolver itself can be reused as an exploitation primitive. By redirecting control flow to the PLT resolver entry and supplying carefully constructed relocation and symbol data, an attacker can cause the dynamic linker to resolve an attacker-chosen function name at runtime. Because the resolver immediately transfers control to the resolved function, the exploit must prepare the required argument registers and stack state before entering the resolver path. This approach removes the need to guess or leak library base addresses.

On modern x86-64 systems, the same idea appears as *ret2dlresolve* [4]. Although the internal data structures differ (for example `Elf64_Rela` and `Elf64_Sym` are used instead of their 32-bit versions), the principle remains the same: redirect execution to the PLT resolver, provide a relocation index (`reloc_arg`) that makes the resolver read a *fake relocation entry* from attacker-controlled writable memory (typically `.bss` or the heap), and have that fake relocation entry point to a *fake symbol entry* and a *fake symbol name string* (e.g., `"system"`). The dynamic linker then follows its normal lookup logic: it uses the fake relocation to find the fake symbol, uses the fake symbol to find the string name, resolves the real function address at runtime, and writes the result into the chosen GOT slot. This approach is more complex than using an address leak, but it avoids the need to know the libc base addresses.

4 Experiments and Results

4.1 Experimental setup

This section presents a practical experiment that validates Krahmer's approach on a modern x86-64 system. The original code-chunk–based technique relies on specific instruction sequences in libc (e.g., within ulimit) that are no longer present due to changes in the library's implementation. There-

fore, we replaced the borrowed code chunks with ROP gadgets extracted from the current libc. The vulnerable program used is Krahmer's sample code and implements a TCP server that contains a stack-based buffer overflow in `handle_connection()`, where an unchecked `read()` allows overwriting the return address. The program was compiled on a 64-bit Linux system with stack canaries, PIE, and ASLR disabled, while NX was enabled. These settings allow us to focus on return-to-libc and ROP behavior, specifically to demonstrate how such techniques bypass non-executable memory protections without interference from other defenses.

4.2 Exploit overview

The exploit, detailed in Algorithm 1, targets the x86–64 System V calling convention, where function arguments are passed in registers. To invoke `system("/bin/sh")`, we follow the same general idea as Krahmer's original attack.

The exploit is implemented in Python using the `pwntools` framework. The goal is to control the execution context so that `system()` is called with a controlled argument. Krahmer achieves this by chaining instruction sequences from different `libc` functions.

In our exploit, the same effect is achieved using ROP gadgets. We extract the offsets of `system()` and the `"/bin/sh"` string from `libc` using `readelf` and `strings`, and locate a suitable `pop rdi; ret;` gadget with `ROPgadget`, along with a standalone `ret;` gadget to ensure stack alignment. These values are combined into a ROP chain that aligns the stack, loads the address of `"/bin/sh"` into `%rdi`, and transfers control to `system()`, thereby bypassing NX without injecting shellcode.

4.3 Results and limitations

When the payload is sent to the server, execution is successfully redirected and an interactive shell is obtained. This confirms that borrowed code chunks remain available in modern libc versions and that return-to-libc attacks on x86-64 systems are still practical.

The exploit assumes ASLR is disabled and therefore relies on fixed library addresses. In current systems, ASLR breaks this assumption. To regain reliability under ASLR, attackers may rely on brute-force techniques, information leaks, or dynamic linker abuse methods such as `dl-resolve`, as described by Wojtczuk. However, brute-force attacks were mainly practical on 32-bit systems and are generally impractical on modern 64-bit systems.

Algorithm 1: ROP exploit against vulnerable server

Data: Buffer overflow in `read()`, libc base = `0x7ffff7c00000`
Result: Shell access via `system("/bin/sh")`

Compute addresses
`system` ← `libc_base` + `0x50d70`
`binsh` ← `libc_base` + `0x1d8678`
`pop_rdi` ← `libc_base` + `0x2a3e5`
`ret` ← `libc_base` + `0x29139`
Build payload
`payload` ← 'A' × 1024 + 'B' × 8
`payload` ← `payload` + `ret`
`payload` ← `payload` + `pop_rdi` + `binsh`
`payload` ← `payload` + `system`
Exploit
Send payload to server → shell obtained

5 Conclusion

Modern systems use several protections to make buffer overflow exploitation more difficult, including non-executable memory, register-based calling conventions, and address randomization. These defenses prevent many traditional attacks, but they do not completely stop exploitation.

Instead, attackers adapt by reusing existing code already present in the program or its libraries and by carefully controlling program state. Even when memory addresses change between executions, attackers can still achieve reliable behavior by using information leaks, repeated attempts, or runtime mechanisms such as dynamic symbol resolution.

Overall, this shows that current protections mainly increase the difficulty of exploitation rather than eliminating it. More robust defenses require mechanisms that restrict how control flow can be redirected, not just where code and data are placed in memory.

References

[1] Solar Designer (Alexander Peslyak). Getting around non-executable stack (and fix). Bugtraq mailing list, 1997.

[2] Sebastian Krahmer. x86-64 buffer overflow exploits and the borrowed code chunks exploitation technique, 2005.

[3] Rafal Wojtczuk. The advanced return-into-lib(c) exploits: Pax case study. Phrack Magazine, Volume 0x0b, Issue 0x3a, Phile #0x04 of 0x0e, 2001.

[4] D3vil. ret2dl_resolve x64: Exploiting dynamic linking procedure in x64 elf binaries, May 2020.